

Data Structure

05. 프로세스 동기화

5주차: Process Synchronization

? 오늘 할거

1

공유 데이터 접근

여러 스레드가 동일 자원에 접근

2

Race Condition

Context Switch로 순서 꼬임 발생

3

Critical Section

보호해야 할 코드 영역 지정

4

Lock / Mutex

상호 배제로 안전한 동시성 완성

목차 INDEX

01 동시성

- Concurrency
- Parallelism

02 동시성 문제

- 공유 자원
- Shared State
- Race Condition
- Lost Update
- 데이터 정합성 문제

03 프로세스 동기화

- Critical Section
- Lock & Mutex
- Semaphore

01

동시성

Concurrency

Review: Thread & CPU Scheduler

3주차: Thread의 자원 공유

- 한 Process 내의 여러 Thread는 Heap (자료구조 8주차때 할거임)과 공유 데이터를 함께 공유
- 메모리를 공유하므로 통신이 빠르고 효율적이지만, 동시 접근 문제가 있음

4주차: CPU Scheduler & Context Switch

- OS의 CPU Scheduler는 언제든지 Thread A를 멈추고 Thread B로 Context Switch 가능
- 스레드의 실행 순서와 교체 시점은 개발자가 예측할 수 없음

근데 여러 스레드나 프로세스가
같은 파일을 동시에 수정하면 어떻게 되는거지?

동시성: Concurrency



논리적인 동시 실행 (Interleaving)

- 싱글 코어 CPU에서도 여러 작업을 번갈아가며 빠르게 실행하여 동시에 실행되는 것처럼 보이게 만듦



Context Switch를 통해 동시성을 구현할 수 있음
(3주차 내용)

Concurrency & Parallelism

구분	동시성 (Concurrency)	병렬성 (Parallelism)
핵심 개념	여러 작업을 번갈아 처리 (Interleaving)	여러 작업을 시각적으로 완전히 동시에 처리
필요 하드웨어	Single-core CPU에서도 가능	Multi-core CPU 필수
관점	프로그램의 논리적 구조 (Structure)	하드웨어의 물리적 실행 (Execution)
주요 목적	응답성 (Responsiveness) 향상 및 대기시간 최소화	연산 속도 (Performance) 극대화

02

동시성 문제

Concurrency Problems

동시성 문제

메모리 영역의 공유

- 스레드들은 각자의 Stack 영역을 가지지만, **Heap 영역과 전역 변수(Global Variables)**를 공유
- 여러 스레드가 읽기만(Read-only) 할 때는 아무런 문제가 발생하지 않음

Shared State의 수정

- 최소 하나 이상의 스레드가 공유 데이터를 수정(Write)할 때 문제가 발생함
- 실행 순서에 따라 데이터 상태가 변하므로 **예측 불가능한 결함**이 생길 수밖에 없음

동시성 문제



공동계좌 잔액: 10만원



두 명이 동시에 출금하는 상황

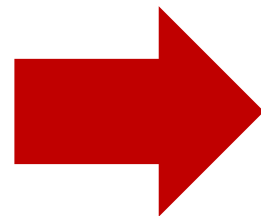
동시성 문제



각각 10,000원씩 출금



공동계좌 잔액: 10만원
→ 동시에 출금한 순간 10,000 출금 실행
→ 계좌에 남은 금액은 총 90,000원



뭔가 이상한듯

Lost Update

- 여기서 1명분의 작업이 실행되지 않음
- 은행 내부 코드에서 A가 돈을 출금하던 중 Context Switch가 일어나 B의 출금 요청을 처리했기 때문
- 이런 것을 Lost Update라고 함



각각 10,000원씩 출금

02 동시성 문제 Concurrency Problems

1

공유 데이터 접근

여러 스레드가 동일 자원에 접근

2

Race Condition

Context Switch로 순서 꼬임 발생

3

Critical Section

보호해야 할 코드 영역 지정

4

Lock / Mutex

상호 배제로 안전한 동시성 완성



Race Condition

- 아까의 경우와 같이 두 개 이상의 스레드 또는 프로세스가 충분히 공유 자원에 동시에 접근할 수 있음
- 이 때 실행 순서에 따라 결과가 달라지게 되는데, 이를 Race Condition이라 함
- 우리는 이 버그를 해결하는 방법을 배울거임



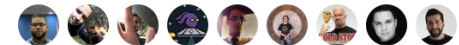
I Am Developer
@iamdeveloper

Follow

Knock knock
Race condition
Who's there?

12:07 PM - 11 Nov 2013

2,504 Retweets 1,013 Likes



38 2.5K 1.0K



Race Condition의 주요 문제



재현 불가능

테스트할 때는 정상 동작하다가,
실제 고객 서비스 중 예측 불가능하게
터지는 버그가 될 가능성이 높음



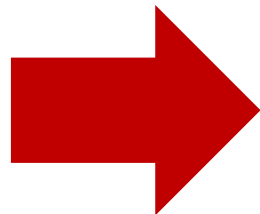
디버깅의 어려움

디버거를 붙이거나 print문을 찍는
타이밍 변화만으로도 Context Switching
순간이 바뀌어 증상이 사라짐



데이터 오염

금융, 결제, 수량 관리 시스템에서
심각한 재산적 손실과 시스템 신뢰성
상실을 야기할 수 있음



프로세스 동기화로 해결
Process Synchronization

02 동시성 문제 Concurrency Problems

1

공유 데이터 접근

여러 스레드가 동일 자원에 접근



2

Race Condition

Context Switch로 순서 꼬임 발생



3

Critical Section

보호해야 할 코드 영역 지정

4

Lock / Mutex

상호 배제로 안전한 동시성 완성

03

프로세스 동기화

Process Synchronization

Critical Section

Critical Section 이란?

- 둘 이상의 스레드가 공유 자원에 접근하여 데이터 수정이 일어나는 코드 영역임
- 아까 봤던 공동계좌가 바로 Critical Section의 예시임



동기화(Synchronization)의 목적

- Critical Section에는 오직 한 번에 하나의 스레드만 들어갈 수 있도록 통제하는 것
- 실행 순서를 일관되게 맞추어 데이터의 정합성 보장
- 여기서 정합성이란 앞뒤가 맞고, 올바른 상태를 유지하는 것이라 생각하면 됨

Mutual Exclusion



Mutual Exclusion

한 스레드가 Critical Section에서 작업 중이라면,
다른 모든 스레드는 접근이 금지되어
대기해야 한다는 원칙

Mutual Exclusion



하나의 스레드가 Critical Section에서 작업 중이면
다른 모든 스레드는 해당 Critical Section에 접근이 불가함

03 프로세스 동기화 Process Synchronization

1

공유 데이터 접근

여러 스레드가 동일 자원에 접근



2

Race Condition

Context Switch로 순서 꼬임 발생



3

Critical Section

보호해야 할 코드 영역 지정

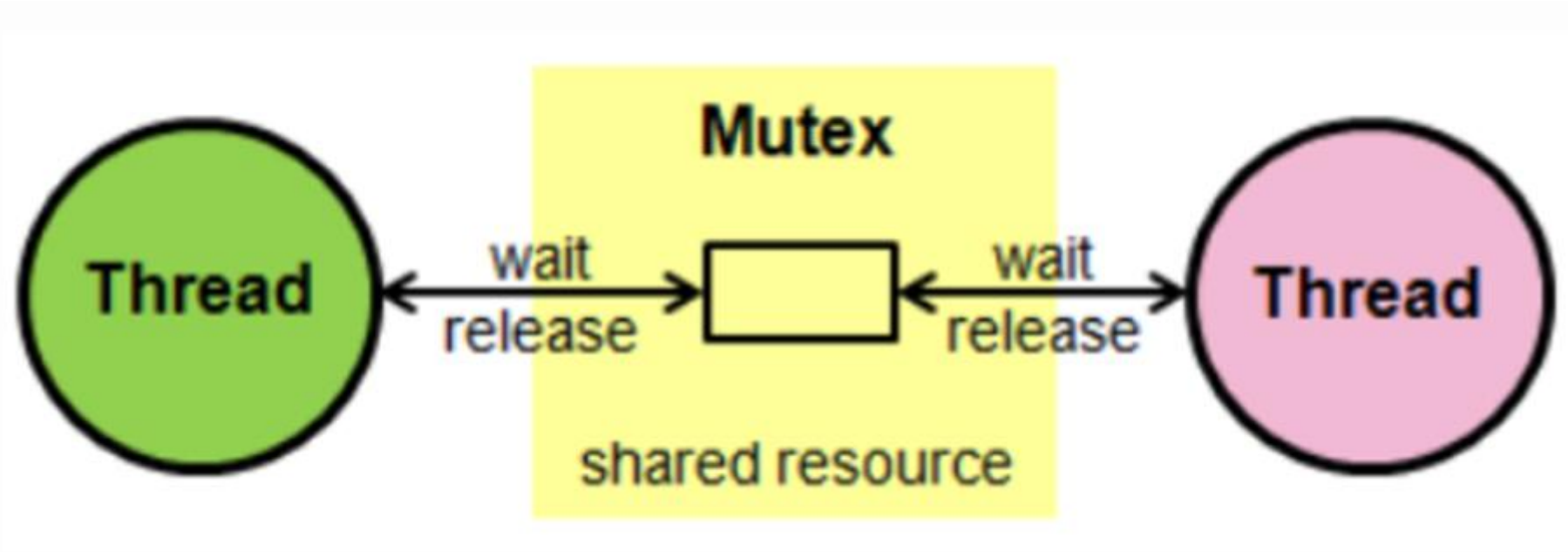


4

Lock / Mutex 등

상호 배제로 안전한 동시성 완성

Mutex: MUTual EXclusion



스레드는 Critical Section(화장실)에 진입하기 전 문을 잠금 (Lock이라 부름)
만약 이미 Lock 된 Critical Section이라면 락 풀릴때까지 대기 (Block이라 부름)
볼일 끝나면 잠금 해제 (Release라 부름)

Mutex: MUTual EXclusion

Mutex (MUTual EXclusion)의 기본 동작

- **acquire:** Critical Section에 진입하기 전 자물쇠를 잠금
이미 잠겨있다면 풀릴 때까지 대기(Block)
- **release:** 작업을 마친 후 자물쇠를 풀어 대기 중인
다른 스레드가 들어올 수 있게 함

소유권(Ownership)의 개념

- Mutex는 열쇠를 쥔 자(소유자) 개념이 명확한 편임
- Lock을 걸어둔(acquire한) 바로 그 스레드만이
Lock을 해제(release)할 수 있음

Mutex 에서 Lock 사용 시 단점



1. 성능 저하 (Overhead)

- Lock을 획득하고 해제하는 데 CPU 연산 비용이 소모됨
- 병렬성이 줄어들고 스레드가 대기(Blocking)하므로 프로그램이 **직렬화(Sequential)**되어 느려질 수 있음
- 무조건 하나의 스레드만 들어가서 작업하므로 속도가 좀 느려질 수 있을듯



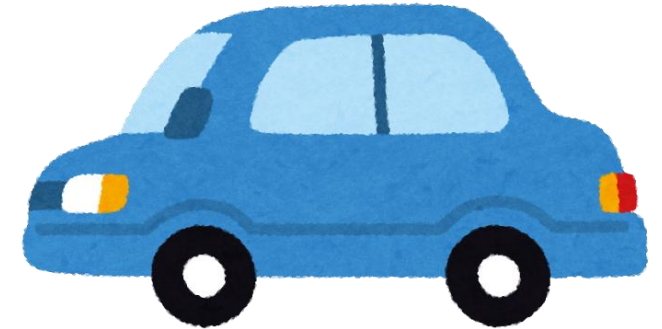
2. 교착 상태 (Deadlock)

- 두 개 이상의 스레드가 서로가 가진 Lock이 풀리기를 영원히 기다리는 상태
- 6주차에서 배울거임

Semaphore



Block 상태



빈 방에 무조건 1명만 들어가야 했던 Mutex 와 달리
Semaphore의 빈 방에는 N명까지 들어갈 수 있음
주차 공간이 2개인 주차장에서 3번째 차는
공간 날때까지 기다리는거 생각하면 바로 이해될걸

Mutex & Semaphore

비교 항목	Mutex	Semaphore
동시 접근 가능 수	오직 1개 스레드만 접근 가능	N개의 스레드 접근 가능 (설정값)
소유권 (Ownership)	있음 (Lock을 건 스레드만 해제 가능)	없음 (다른 스레드가 signal/release 가능)
비유	1인용 화장실 열쇠	N개의 주차 공간이 있는 주차장
주 주요 용도	임계 구역 내 변수/데이터 독점 보호	한정된 자원(DB 커넥션 풀 등) 개수 제어

03 프로세스 동기화 Process Synchronization

1

공유 데이터 접근

여러 스레드가 동일 자원에 접근



2

Race Condition

Context Switch로 순서 꼬임 발생



3

Critical Section

보호해야 할 코드 영역 지정



4

Lock / Mutex 등

상호 배제로 안전한 동시성 완성

